



Práctica final - Restaurante



EL QUINTO PINO



Álvaro Martínez Cuadros



C.E.S. Vega Media

ÍNDICE

ÍNDICE	1
Sprint 1 - Fundamentos del Sistema y Roles	2
Estructura básica del sistema y diseño responsive	2
Base de datos funcional	2
Sistema de autenticación y registro	3
Roles diferenciados	3
Seguridad	3
Creación y modificación de categorías, productos y stock de la parte de encargado	4
Sprint 2 - Gestión de Mesas y toma de pedidos.	5
Interfaz para los clientes e implementación de control de stock	5
Control de stock	6
Notas específicas	6
Filtro de la carta por búsqueda	6
Funcionalidad del camarero	7
Controlar que el cliente no pueda seleccionar otra mesa antes de pagar	8
Lógica de la carta	9
Lógica del carrito	10
Lógica de pedir	11
Sprint 3 - Generación de cuentas y cierre de Mesas	13
Generación automática de cuentas en formato PDF	13
Cobrar el pedido	15
Ver factura del cliente	16
Creación de tickets	17
Detalles del ticket	18
Sprint 4 - Funciones Administrativas y Gestión del Menú	19
Estadísticas -- Investigación	19
Inconvenientes con Chart.js	20
Generación de estadísticas	21
Top 3 productos más vendidos	22
Tickets de cocina	23
Capacidades del rol Encargado	24
Añadir personal	25
Añadidos extra	26
Bibliografía	27

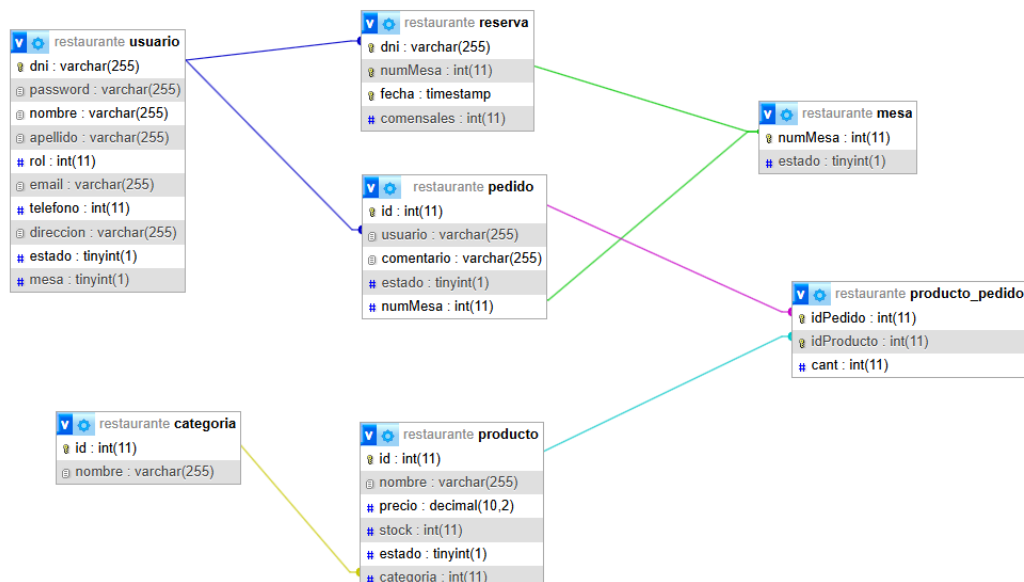
Sprint 1 - Fundamentos del Sistema y Roles

Estructura básica del sistema y diseño responsive

- He realizado desde 0, una plantilla realizada con Bootstrap y CSS propio.
- He verificado que la plantilla sea responsive, con las facilidades de Bootstrap.
- Estructura del proyecto (por ahora):



Base de datos funcional



Sistema de autenticación y registro

- Desde **index.php** se puede iniciar sesión, y desde **registro.php** se pueden registrar nuevos clientes. Una vez logueado, el sistema redirige al usuario según su rol.

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $dni = $_POST["dni"];
    $pw = $_POST["pw"];

    $consulta = "SELECT * FROM usuario WHERE dni = '$dni' AND password = '$pw'";

    $result = mysqli_query($conn, $consulta);
    $row = mysqli_num_rows($result);
    $row2 = mysqli_fetch_array($result);

    if ($row == 1) {

        $_SESSION["usuario"] = $row2["nombre"];
        $_SESSION["dni"] = $dni;
        $_SESSION["rol"] = $row2["rol"];

        if ($row2["estado"] == 1) {
            $mensaje = "Has sido bloqueado.<br>Contacta con un administrador.";
        } else {
            if ($row2["rol"] == 0) {
                header("LOCATION: cliente/consulta.php");
                $mensaje = "Soy rol 0";
            } else if ($row2["rol"] == 1) {
                header("LOCATION: camarero/camarero.php");
                $mensaje = "Soy rol 1";
            } else if ($row2["rol"] == 2) {
                header("LOCATION: encargado/encargado.php");
                $mensaje = $row2["rol"];
            }
        }
    } else if ($row == 0) {
        $mensaje = "Usuario o contraseña incorrectos.";
    }
}
```

Roles diferenciados

-  /cliente/ → rol: 0
-  /camarero/ → rol: 1
-  /encargado/ → rol: 2

Cada uno tiene su propia zona, navegación y cierre de sesión.

Seguridad

Todos los archivos revisan la sesión y el rol con el archivo **seguridad.php**. Si un usuario intenta acceder donde no debe, se le redirige al inicio.

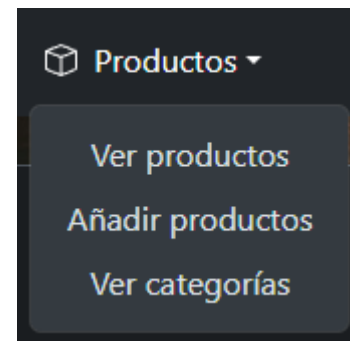
```
<?php

if (!isset($_SESSION["usuario"]) || $_SESSION["rol"] != 0) {
    header("LOCATION: ../index.php");
    exit;
}
```

Creación y modificación de categorías, productos y stock de la parte de encargado

El encargado puede:

- ✓ Añadir productos
- ✓ Editar productos
- ✓ Bloquear/desbloquear productos
- ✓ Añadir categorías
- ✓ Editar categorías



Bienvenido, Álvaro
Consulta tus productos

#	Nombre	Precio	Stock	Categoría	Editar	
1	Agua mineral	1.40	100	Bebidas	<button>Editar</button>	<button>Bloquear</button>
2	Coca-Cola	2.00	80	Bebidas	<button>Editar</button>	<button>Bloquear</button>
3	Fanta Naranja	2.00	75	Bebidas	<button>Editar</button>	<button>Bloquear</button>
4	Cerveza	2.50	60	Bebidas	<button>Editar</button>	<button>Bloquear</button>
5	Té frío	1.80	50	Bebidas	<button>Editar</button>	<button>Bloquear</button>
6	Patatas fritas	3.00	40	Aperitivo	<button>Editar</button>	<button>Bloquear</button>
7	Aceitunas	2.50	35	Aperitivo	<button>Editar</button>	<button>Bloquear</button>

Si se bloquea un producto, éste no será visible para el cliente.

Añadir un producto

Nombre

Precio

Stock

Bloqueado ☐ No ☐ Categoría

Categorías

Nombre	Editar
Bebidas	Editar
Aperitivo	Editar
Primero	Editar
Segundo	Editar
Postres	Editar
Cafés	Editar
Regalos	Editar

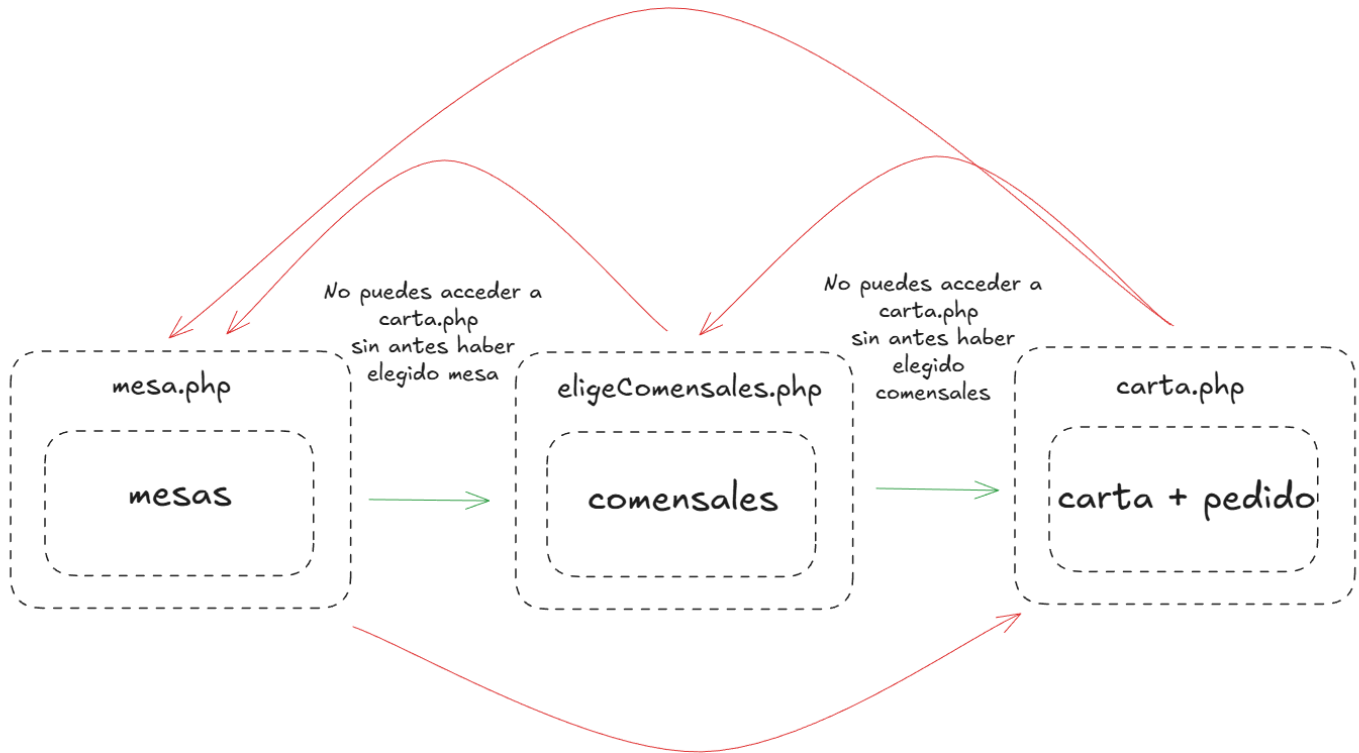
Añade una categoría

Nombre

Sprint 2 - Gestión de Mesas y toma de pedidos.

Interfaz para los clientes e implementación de control de stock

El flujo para los clientes es el siguiente:



En **mesa.php**:

- Consulta a la base de datos las mesas que existen y se pintan sobre el diseño en un sistema *grid* diseñado exclusivamente por mí.
- Se puede seleccionar una mesa, esto redirige a **eligeComensales.php**.
- Al realizarlo, **automáticamente se realiza un insert** en la base de datos en la **tabla reserva** poniendo a **cero** los comensales.

En **eligeComensales.php**:

- Al poner un número de comensales, como mínimo 1 y máximo 15, se realiza un update en el insert anteriormente mencionado sobre los comensales.
- Teniendo todos los campos cubiertos, podremos entrar a **carta.php**

En **carta.php**:

- Tenemos 2 *cards*, en 1 tenemos la carta, y en otra el pedido actual.
 - **Carta**: Podremos ver la carta **siempre y cuando tengamos stock y el producto no esté bloqueado** por el encargado. También se puede filtrar por categorías.
 - **Pedido**: Podemos ver el pedido, actualizando las cantidades hasta el límite del stock que haya en ese momento y podemos ver el precio de todo el completo.
- Al enviar el pedido, hacemos un **insert** en la tabla pedidos ocupando el pedido completo del cliente.

Control de stock

He controlado el stock, restándose según la cantidad del pedido.

```
// restamos el stock de la cantidad
$stock = mysqli_query($conn, "SELECT stock FROM producto WHERE id = $idProducto");
$rowStock = mysqli_fetch_array($stock);
$stock = $rowStock["stock"] - $cant;
mysqli_query($conn, "UPDATE producto SET stock = '$stock' WHERE id = '$idProducto'");
```

Se verifica previamente añadiendo un máximo en el input.

```
<input type="number"
name="cantidades[<?php echo $idProd ?>]"
value="<?php echo $cant ?>"
min="1" max="<?php echo $rowStock["stock"] ?>"
class="form-control text-center">
<small class="small text-muted text-start">Máx: <?php echo $rowStock["stock"] ?></small>
```

El máximo está seteado al máximo stock disponible, lo cual no se puede poner más de lo que exista, además, se añade un texto de ayuda, donde indica la cantidad máxima que el usuario puede pedir.

Notas específicas

He añadido un '`<textarea>`' en cada apartado de productos del pedido, pudiendo así, generar un comentario por cada producto.



Pedidos

Pedido #4 Pendiente

Mesa 8

Producto	Precio	Cantidad	Comentario	Subtotal
Coca-Cola	2.00 €	3	1 zero	6.00 €
Aceitunas	2.50 €	1		2.50 €
Croquetas caseras	4.50 €	1	jamón	4.50 €
Calamares a la romana	5.00 €	1		5.00 €
Pollo al horno	9.00 €	2		18.00 €
Salmón a la plancha	12.00 €	1		12.00 €
Total				48.00 €

Filtro de la carta por búsqueda

He añadido un filtro en la carta que filtra por **categorías**, así es más fácil buscar los productos.



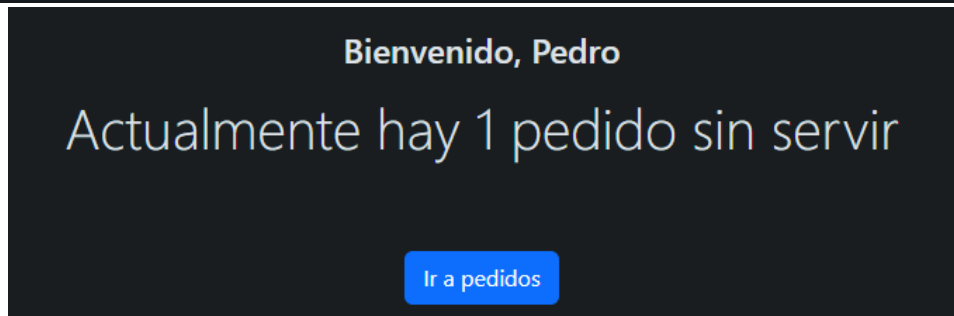
```
<!-- FILTRO POR CATEGORÍA -->
<form method="get" action="" class="mb-3">
  <div class="row g-2 align-items-center">
    <div class="col-8">
      <select name="categoria" class="form-select">
        <?php
        $categorias = mysqli_query($conn, "SELECT * FROM categoria");
        while ($cat = mysqli_fetch_array($categorias)) {
          $idCat = $cat["id"];
          $nombreCat = $cat["nombre"];
          if (isset($_GET["categoria"]) && $_GET["categoria"] == $idCat) {
            print("<option value='$idCat' selected>$nombreCat</option>");
          } else {
            print("<option value='$idCat'>$nombreCat</option>");
          }
        }
        ?>
      </select>
    </div>
    <div class="col-4 text-end">
      <button type="submit" class="btn btn-primary w-100">Filtrar</button>
    </div>
  </div>
</form>
```

Funcionalidad del camarero

Al introducir sus credenciales, el camarero entra en **camarero.php**, una página de inicio donde se indica si hay o no pedidos pendientes. **Esta página se recarga cada 5 segundos** por esta función de JavaScript, así, podremos ver en tiempo real si se ha registrado un nuevo pedido.

```
<script>
  // recargamos la página por si hay pedidos, que aparezcan en el home
  function refrescarPagina() {
    location.reload();
  }

  setTimeout(refrescarPagina, 5000);
</script>
```



Una vez que haya un pedido, si accedemos a **pedidos.php**, nos encontramos con lo siguiente:

Producto	Cantidad	Comentario	Servido
Coca-Cola	3	1 zero	☒
Aceitunas	1		☒
Croquetas caseras	1	jamón	☒
Calamares a la romana	1		☐
Pollo al horno	2		☐
Salmón a la plancha	1		☐
Pedido servido			

Producto	Cantidad	Comentario	Servido
Agua mineral	1		☒
Coca-Cola	1		☒
Fanta Naranja	1		☒
Cerveza	3		☐
Patatas fritas	1		☐
Pedido servido			

Donde obligatoriamente debemos de marcar todos los **check**, y después, darle a Pedido servido y se quitará de la vista del camarero, esperando a que el encargado lo cobre.

Producto	Cantidad	Comentario	Servido
Coca-Cola	3	1 zero	☒
Aceitunas	1		☒
Croquetas caseras	1	jamón	☐
Calamares a la romana	1		☐
Pollo al horno	2		☐
Salmón a la plancha	1		☐
Pedido servido			

Después de haber servido el producto, al cliente le aparecerá el siguiente texto de ayuda:

Pedido #1 Servido				
Mesa 2				
Producto	Precio	Cantidad	Comentario	Subtotal
Agua mineral	1.50 €	1		1.50 €

Si el cliente vuelve a realizar otro pedido, este se volverá a poner como Pendiente.

Controlar que el cliente no pueda seleccionar otra mesa antes de pagar

Este control se ha implementado mediante una **lógica por fases**, gestionada a través de **variables de sesión booleanas**.

Cada fase del proceso, desde el inicio de sesión hasta la finalización del pedido, cuenta con su propio control, lo que permite aplicar las restricciones necesarias en cada momento.

En **mesa.php**, cuando el cliente selecciona una mesa, le lleva a **reserva.php**, donde se establece la reserva y la variable de sesión **haElegidoMesa** se establece en **true**. De esta forma, el usuario no puede volver a esta sección hasta que la mesa haya sido pagada.

```
$m = $_GET["mesa"];

$_SESSION["mesa"] = $m;
$_SESSION["haElegidoMesa"] = true;

$fechaAct = date("Y-m-d H:i:s");

$consulta = "INSERT INTO reserva (dni, numMesa, fecha) VALUES ('$usuario', '$m', '$fechaAct')";
$result = mysqli_query($conn, $consulta);

mysqli_query($conn, "UPDATE mesa SET estado = 0 WHERE numMesa = '$m'");
```

En **eligeComensales.php**, al indicar el número de comensales, se activa la variable de sesión **haElegidoComensales**, también con valor **true**, impidiendo el acceso nuevamente a esta fase mientras la mesa siga pendiente de pago.

```
// realizamos update de los comensales
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $comensales = $_POST["comensales"];

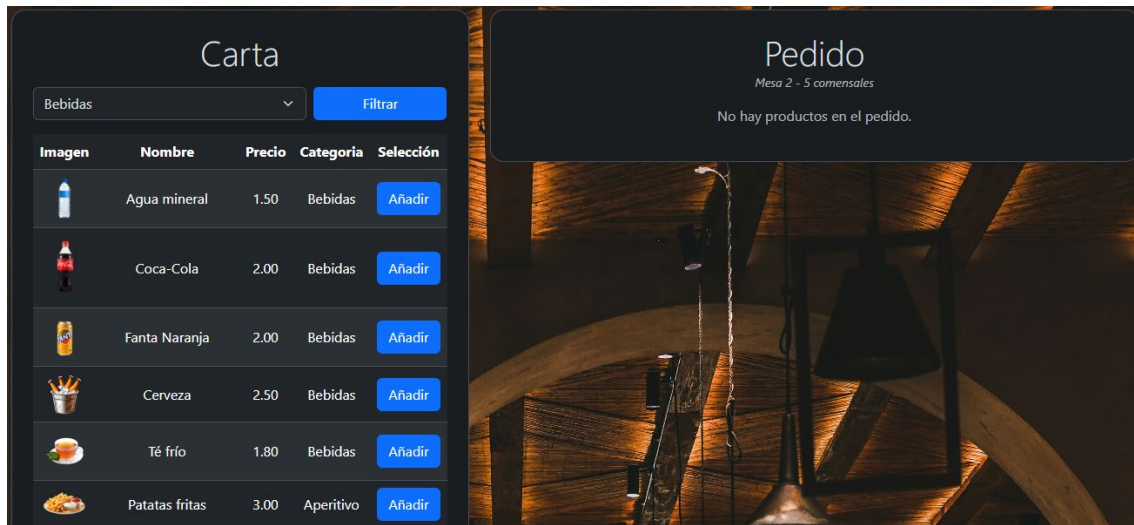
    mysqli_query($conn, "UPDATE reserva SET comensales = '$comensales' WHERE dni = '$dni' AND comensales = 0");

    $_SESSION["haElegidoComensales"] = true;
    header("LOCATION: carta.php");
    exit;
}
```

Finalmente, cuando el encargado marca la mesa como **"Pagada"**, ambas variables de sesión se liberan y **la reserva queda anulada**, permitiendo que el cliente pueda volver a gestionar una nueva mesa desde el inicio del proceso.

Lógica de la carta

Para la lógica de la carta, nos aparece una lista con todos los productos en una card, con la imagen, el nombre del producto, el precio, y un botón de Añadir.



En esta card, se ha añadido la funcionalidad de filtrar por categorías; cada vez que pones una categoría, se hace una selección de los pedidos de esa categoría.



```
// filtrado por categoria
if (isset($_GET["categoria"])) {
    $cat = $_GET["categoria"];
    $consulta = "SELECT * FROM producto WHERE categoria = '$cat'";
    $result = mysqli_query($conn, $consulta);
} else {
    $consulta = "SELECT * FROM producto";
    $result = mysqli_query($conn, $consulta);
}
```

Cuando pulsamos en añadir, añadimos el producto a la lista de pedido:

```
// si el carrito NO está inicializado, lo inicializamos
if (!isset($_SESSION['carrito'])) {
    $_SESSION['carrito'] = [];
}

// agregamos la id del producto al carrito y recargamos la página
if (isset($_GET['id'])) {
    $id = $_GET['id'];
    if (!isset($_SESSION['carrito'][$id])) {
        $_SESSION['carrito'][$id] = 0;
    }
    $_SESSION['carrito'][$id] += 1;
    header("LOCATION: carta.php");
    exit;
}
```

Añade el producto a la variable de sesión 'carrito', que a su vez es una array asociativa.

Lógica del carrito

Mostramos en una tabla la sesión del carrito con los productos añadidos y los campos necesarios para su correcta inserción en la base de datos.

```
<!-- CARRITO -->
<form action="" method="post">
  <?php if (empty($_SESSION['carrito'])) { ?>
    <p class="text-center text-muted">No hay productos en el pedido.</p>
  <?php } else { ?>
    <div class="table-responsive">
      <table class="table table-hover table-striped align-middle align-items-center">
        <thead>
          <tr>
            <th>Producto</th>
            <th class="text-center">Precio</th>
            <th class="text-center">Cantidad</th>
            <th class="text-center">Comentario</th>
            <th class="text-center">Subtotal</th>
            <th class="text-center">Acciones</th>
          </tr>
        </thead>
        <tbody>
          <?php
          $total = 0.0;
          foreach ($_SESSION["carrito"] as $idProd => $cant) {
            // se recupera nombre y precio del producto
            $consulta2 = mysqli_query($conn, "SELECT nombre, precio FROM producto
            WHERE id = $idProd");
            $producto = mysqli_fetch_assoc($consulta2);

            $nombre = $producto['nombre'];
            $precio = $producto['precio'];
            $subtotal = $precio * $cant;
            $total += $subtotal;

            echo "<tr>";
            echo "<td>$nombre</td>";
            echo "<td class='text-center'>" . number_format($precio, 2) . " €</td>";
            echo "<td class='text-center' style='max-width:120px;'>";
```

He añadido 3 botones:



Donde **vaciar** (\$GET), hace que el carrito se quede vacío, **actualizar cantidades** (\$POST) coge las cantidades de la sesión 'carrito' y **pedir** (\$POST) crea un pedido con los productos que tenga el carrito más las cantidades.

```
// actualizamos las cantidades cuando pulsamos varias veces en 'Añadir' o el botón de actualizar
if (isset($_POST["cantidades"])) {
  foreach ($_POST['cantidades'] as $id => $cantidad) {
    $_SESSION['carrito'][$id] = $cantidad;
  }
  header("LOCATION: carta.php");
  exit;
}
```

(En esta fase he utilizado apoyo de ChatGPT, pero únicamente después de intentar desarrollar el código por mi cuenta y emplearlo posteriormente para corregir los errores.)

Lógica de pedir

Al darle click a 'Pedir'; en el propio post de **carta.php** se hace la inserción en la tabla **pedido**.

```
// por si el usuario sale de la sesión, evitamos que pierda el pedido asignado
if (!isset($_SESSION['idPedido'])) {
    $pedido = mysqli_query($conn, "SELECT * FROM pedido WHERE usuario = (SELECT dni FROM reserva WHERE dni = '$dni') AND estado != 2");

    $rowPedido = mysqli_num_rows($pedido);
    // si ya existe un pedido con ese DNI y esa MESA
    if ($rowPedido > 0) {
        $rowPedido = mysqli_fetch_array($pedido);
        $idPedido = $rowPedido["id"];
        $_SESSION["idPedido"] = $idPedido;
    } else {
        $fechaAct = date("Y-m-d H:i:s");
        $pedido = "INSERT INTO pedido (usuario, fecha, estado, numMesa)
            VALUES('$dni', '$fechaAct', 0, '$mesa')";

        mysqli_query($conn, $pedido);

        // guardamos el id del pedido en la sesión
        $_SESSION['idPedido'] = mysqli_insert_id($conn);
        $idPedido = $_SESSION["idPedido"];
    }
} else {
    $idPedido = $_SESSION["idPedido"];
}
```

También se realiza una inserción de los productos que se tengan en el carrito en la tabla **producto_pedido**.

```
/*
esta consulta es gracias a chatgpt;
LO QUE HACE: si hay 1 pedido y 1 producto en el mismo pedido; le suma la cantidad al producto.
(mucho mejor que la opción de abajo comentada)
*/
$productoPedido = " INSERT INTO producto_pedido (idPedido, idProducto, cant, comentario)
VALUES ('$idPedido', '$idProducto', '$cant', '$coment')
ON DUPLICATE KEY UPDATE
cant = cant + VALUES(cant),
comentario = VALUES(comentario)";

$updateEstado = mysqli_query($conn, "UPDATE pedido SET estado = 0 WHERE id = '$idPedido'");

// reemplazo por la IA
// if ($coment == "") {
//     $productoPedido = "INSERT INTO producto_pedido (idPedido, idProducto, cant)
//     VALUES ('$idPedido', '$idProducto', '$cant')";
// } else {
//     $productoPedido = "INSERT INTO producto_pedido (idPedido, idProducto, cant, comentario)
//     VALUES ('$idPedido', '$idProducto', '$cant', '$coment')";
// }
// }
```

Después de realizar las inserciones, **se saca un ticket en cocina** (esto se verá más adelante) y se lleva a **pedidos.php**, donde se ve el pedido del cliente.

Pedido #1					Pendiente
Mesa 2			2025-11-19 17:50:55		
Producto	Precio	Cantidad	Comentario	Subtotal	
Agua mineral	1.50 €	3		4.50 €	
Total				4.50 €	

Estructura del proyecto (por ahora)

```

restaurantes/                                # carpeta raíz del proyecto
├── .gitattributes                            # configuración de Git
├── index.php                                # página principal de la aplicación
├── README.md                                # documentación del proyecto
├── registro.php                             # formulario de registro de nuevos usuarios
├── styles.css                               # hoja de estilos principal
├── bd/                                       # scripts SQL de la base de datos
│   ├── restaurante.sql                     # copia de respaldo
│   ├── restaurante_02.sql
│   ├── restaurante_03.sql
│   └── restaurante_04.sql                   # base de datos actual
├── camarero/                               # módulo del rol camarero
│   ├── camarero.php                        # página principal del camarero
│   ├── cerrarSesion.php                    # cierre de sesión
│   ├── editarPerfil.php                    # edición de perfil
│   ├── navbar.php                         # barra de navegación
│   ├── pedidos.php                         # lista de pedidos
│   ├── perfil.php                          # página de perfil
│   ├── productoServido.php                 # marcar productos como servidos
│   └── seguridad.php                       # control de acceso
├── cliente/                               # módulo del rol cliente
│   ├── addProductos.php                   # añadir productos al pedido
│   ├── carta.php                          # ver la carta del restaurante
│   ├── cerrarSesion.php                    # cierre de sesión
│   ├── editarPerfil.php                    # editar perfil del cliente
│   ├── eligeComensales.php                 # selección de número de comensales
│   ├── mesa.php                           # asignación o selección de mesa
│   ├── navbar.php                         # barra de navegación del cliente
│   ├── pedidos.php                         # ver pedidos del cliente
│   ├── perfil.php                          # perfil del cliente
│   ├── reserva.php                         # realizar reserva
│   └── seguridad.php                       # control de acceso
├── components/                             # componentes comunes
│   ├── conexion.php                       # conexión a la base de datos
│   ├── header.php                          # cabecera HTML reutilizable
│   └── navbar.php                           # barra de navegación general
├── encargado/                             # módulo del rol encargado
│   ├── addPersonal.php                     # añadir empleados
│   ├── addProductos.php                     # añadir nuevos productos
│   ├── addProductos02.php                  # versión alternativa de añadir productos
│   ├── bloquearPersonal.php                # bloquear empleados
│   ├── bloquearProducto.php                # bloquear productos
│   ├── categorias.php                      # gestión de categorías
│   ├── cerrarSesion.php                    # cierre de sesión
│   ├── desbloquearPersonal.php              # desbloquear empleados
│   ├── desbloquearProducto.php              # desbloquear productos
│   ├── editarCategoria.php                  # editar categorías
│   ├── editarPerfil.php                     # editar perfil del encargado
│   ├── editarPersonal.php                   # editar información del personal
│   ├── editarPersonal02.php                 # versión alternativa de edición de personal
│   ├── editarProducto.php                   # editar productos existentes
│   ├── editarProducto02.php                 # versión alternativa de edición de productos
│   ├── encargado.php                       # página principal del encargado
│   ├── navbar.php                          # barra de navegación del encargado
│   ├── perfil.php                          # perfil del encargado
│   └── seguridad.php                       # control de acceso

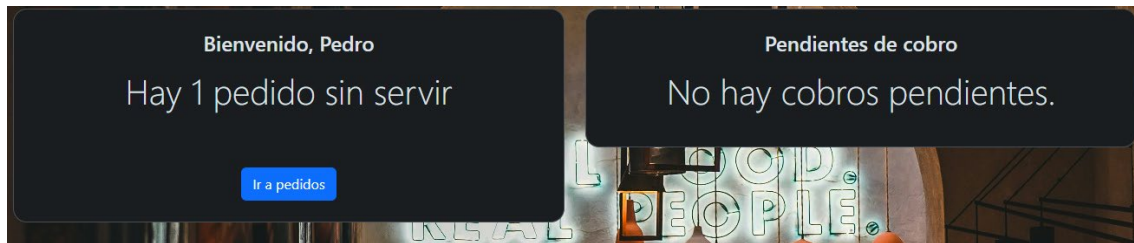
```

Sprint 3 - Generación de cuentas y cierre de Mesas

Generación automática de cuentas en formato PDF.

Para ello, la lógica que he seguido es la siguiente:

El camarero sirve los productos y los productos de ese pedido se marcan como servidos.



Esta página se actualiza automáticamente cada 5 segundos **para realizar la simulación de actualización en línea.**

Una vez entremos a **Ir a pedidos**, podemos marcar los productos por separado si se han servido. No podemos servir el pedido completo si hay partes del pedido que no se han servido.



Si el pedido está marcado como servido, se queda como pendiente de pago. Si está pendiente de pago se puede cobrar, sino no podemos cobrarlo.

Si está pendiente de cobro, nos vamos a la página principal para ver qué sucede.

Bienvenido, Pedro

No hay pedidos sin servir.
¡Buen trabajo!

Pendientes de cobro

Hay pendiente de cobro: 1

[Ir a Cobros](#)

Aparece la **card** como que hay un pendiente de cobro. Si accedemos a **Ir a cobros**:

Cobros pendientes: 1

Pedido #3 - Mesa 2

Producto	Cantidad	Precio
Agua mineral	1	1.50
Cerveza	1	2.50
Patatas fritas	1	3.00
Croquetas caseras	1	4.50
Total:		11.50 €

[Imprimir](#)[Cobrar](#)

Nos aparecen las mesas servidas que están esperando a ser cobradas, en caso de que pidan la cuenta, podemos darle a imprimir para que salga el ticket (*esto se ve un poco más adelante*).

Bienvenido, Pedro

Hay 7 pedidos sin servir

[Ir a pedidos](#)

Pendientes de cobro

Hay pendiente de cobro: 14

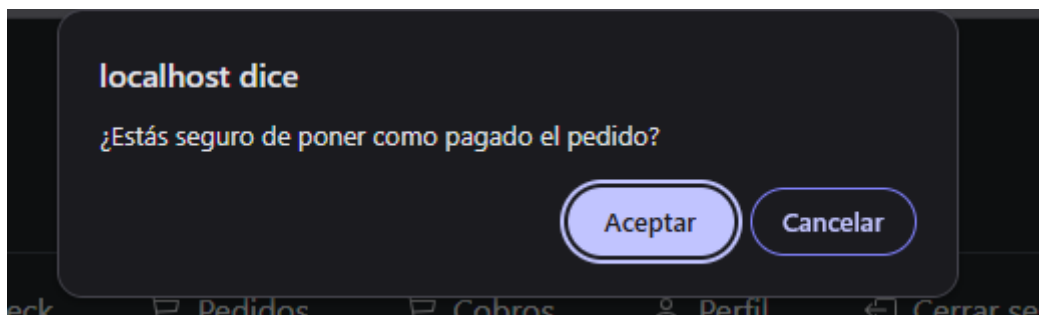
[Ir a Cobros](#)

Cobrar el pedido

Al darle a cobrar, aparece una alerta que he hecho con **JavaScript** muy sencilla, pero creo que necesaria:

```
function pagado(pedido) {  
    let confirmar = confirm("¿Estás seguro de poner como pagado el pedido?");  
  
    if (confirmar) {  
        // aquí va el location.href para pagar y con ello, liberar la mesa y la reserva  
        location.href = "cobrado.php?idPedido=" + pedido;  
    }  
}
```

La ventana emergente es la siguiente



Si le damos a aceptar, nos manda a **cobrado.php** y realizo la lógica de cobrar, que en mi caso es poner el **estado del pedido en 2**. Si el estado del pedido es 2, al cliente se le marca como pagado y al camarero le desaparece del sistema.

Pedido #3					Pagado
Mesa 2		2025-11-20 10:06:17			
Producto	Precio	Cantidad	Comentario	Subtotal	
Agua mineral	1.50 €	1		1.50 €	
Cerveza	2.50 €	1		2.50 €	
Patatas fritas	3.00 €	1		3.00 €	
Croquetas caseras	4.50 €	1		4.50 €	
Total				11.50 €	
					Ver factura

Ver factura del cliente

Para realizar las facturas en formato PDF he instalado la **última versión** con **Composer** de **mPDF**.

```
./composer.json has been updated
Running composer update mpdf/mpdf
Loading composer repositories with package information
Updating dependencies
Lock file operations: 8 installs, 0 updates, 0 removals
- Locking mpdf/mpdf (v8.2.6)
- Locking mpdf/psr-http-message-shim (v2.0.1)
- Locking mpdf/psr-log-aware-trait (v3.0.0)
- Locking myclabs/deep-copy (1.13.4)
- Locking paragonie/random_compat (v9.99.100)
- Locking psr/http-message (2.0)
- Locking psr/log (3.0.2)
- Locking setasign/fpdi (v2.6.4)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 8 installs, 0 updates, 0 removals
- Installing setasign/fpdi (v2.6.4): Extracting archive
- Installing psr/log (3.0.2): Extracting archive
- Installing psr/http-message (2.0): Extracting archive
- Installing paragonie/random_compat (v9.99.100): Extracting archive
- Installing myclabs/deep-copy (1.13.4): Extracting archive
- Installing mpdf/psr-log-aware-trait (v3.0.0): Extracting archive
- Installing mpdf/psr-http-message-shim (v2.0.1): Extracting archive
- Installing mpdf/mpdf (v8.2.6): Extracting archive
2 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
3 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
No security vulnerability advisories found.
Using version ^8.2 for mpdf/mpdf
```

Después de instalarlo y desarrollar el código del PDF **en formato HTML** en **pdf.php** con la librería, hacemos que al darle a 'Ver factura', aparezca el respectivo PDF ya configurado.



El Quinto Pino

Restaurante El Quinto Pino
Ctra. De Mula, 37, Murcia - Alguazas · Tel: 968 620 913 · CIF: 12345678B

Datos del cliente

Nombre: Cliente01 Prueba01
DNI: 3
Email: cliprueb01@quintopino.com
Teléfono: 3

Datos del pedido

Nº Pedido: 2
Fecha: 2025-11-20 08:59:33
Mesa: 9
Estado: Pagado

Detalle del consumo

Producto	Cantidad	Subtotal
Agua mineral	3	4.50 €
Croquetas caseras	1	4.50 €
Pollo al horno	1	9.00 €
Lomo con patatas	1	8.50 €
Hamburguesa completa	1	9.50 €
Brownie con helado	1	4.50 €
Base Imponible:		33.47 €
IVA (21%)		7.03 €
Total		40.50 €

Gracias por su visita a El Quinto Pino.
Esperamos volver a verle pronto.

Factura generada automáticamente como parte de la práctica de 2º DAW.

Ha sido creado con tablas, ya que **mPDF** no acepta *Bootstrap*, *flex* ni *grid*.

Creación de tickets

Para la creación de tickets, he instalado la última versión de Mike42.

```
./composer.json has been created
Running composer update mike42/escpos-php
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
- Locking mike42/escpos-php (v4.0)
- Locking mike42/gfx-php (v8.6)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
- Installing mike42/gfx-php (v8.6): Extracting archive
- Installing mike42/escpos-php (v4.0): Extracting archive
1 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
No security vulnerability advisories found.
Using version ^4.0 for mike42/escpos-php
```

Una vez descargado, he utilizado el ejemplo que tuvimos de *classroom* y he utilizado las consultas que necesito para sacar los datos de **producto_pedido** y del **pedido** en cuestión para su correcta impresión.

Las impresiones de tickets aparecen en dos ocasiones:

- Cuando el usuario realiza un pedido, se envía un ticket a cocina (*Sprint 4*).



- Cuando el camarero accede a imprimir el ticket, aparece el ticket de toda la mesa.



Detalles del ticket

Una de las problemáticas que he encontrado con el ticket es que, al poner tildes, borraba un espacio. He tenido que quitar las tildes del ticket para evitar este error con la siguiente función:

```
// quita las tildes en el ticket, evita errores de quitar espacios
function quitaTildes($texto)
{
    $originales = array(
        "á", "é", "í", "ó", "ú"
    );

    $reemplazos = array(
        "a", "e", "i", "o", "u"
    );

    return str_replace($originales, $reemplazos, $texto);
}
```

Después, otro problema que encontré fueron los espaciados y cómo centrar correctamente las unidades y los precios, ya que, con diferentes cantidades de unidad y diferentes cantidades de euros, se descentraba todo.

Para ello, usé esta función:

```
// reservamos espacios; 20 pegados a la izquierda, 3 para la cantidad y 10 para el precio
$printer->text(sprintf("%-20s %3s %10s EUR\n", $nombre, $cantidad, $precioFormateado));
```

Así, siempre se reservan los mismos espacios para todo y no crea desajustes.

```
// cabecera de la tabla
$printer->setJustification(Printer::JUSTIFY_LEFT);
$printer->text("PRODUCTO          UDS      IMPORTE\n");
$printer->text(str_repeat("=", 39) . "\n");

// líneas
while ($linea = mysqli_fetch_array($consultaLineas)) {
    $idProducto = $linea["idProducto"];
    $cantidad = $linea["cant"];
    // saco el precio
    $precioConsulta = mysqli_query($conn, "SELECT * FROM producto WHERE id = '$idProducto'");
    $row = mysqli_fetch_array($precioConsulta);
    $precio = $row["precio"];
    $nombre = $row["nombre"];
    $nombre = quitaTildes($nombre);
    $precio = $precio * $cantidad;
    $precioFormateado = number_format($precio, 2);
    $total += $precioFormateado;

    // reservamos espacios; 20 pegados a la izquierda, 3 para la cantidad y 10 para el precio
    $printer->text(sprintf("%-20s %3s %10s EUR\n", $nombre, $cantidad, $precioFormateado));
}

// total
$IvaTipo = 0.21; // 21%
$base_imponible = $total / (1 + $IvaTipo);
$cuota_iva = $total - $base_imponible;

$printer->text(str_repeat("=", 39) . "\n");
$printer->setJustification(Printer::JUSTIFY_LEFT);
$printer->text("Base Imponible: " . number_format($base_imponible, 2) . " EUR\n");
$printer->text("IVA (21%): " . number_format($cuota_iva, 2) . " EUR\n");
$printer->text(str_repeat("-", 39) . "\n");
$printer->setEmphasis(true);
$printer->text("TOTAL: " . number_format($total, 2) . " EUR\n");
$printer->setEmphasis(false);
```

Sprint 4- Funciones Administrativas y Gestión del Menú

Estadísticas -- Investigación

He realizado una investigación exhaustiva sobre una **gráfica visual** para implementarla con **JavaScript**:

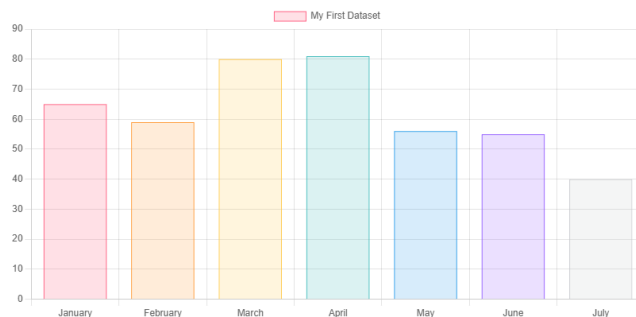
MDBootstrap: Librería adicional de Bootstrap, donde existen componentes mejorados y componentes extras. Intenté coger de ahí un componente de gráficas, pero finalmente me decidí por **Chart.js**

Chart.js: Librería de **JavaScript** para realizar gráficos en tiempo real, con unas animaciones que quedan muy bien, que siguiendo su documentación, vi que *'no era tan difícil de implementar'*.

Al entrar en la documentación, la visual me llamó muchísimo la atención.

Bar Chart

A bar chart provides a way of showing data values represented as vertical bars. It is sometimes used to show trend data, and the comparison of multiple data sets side by side.



Estaba pensando justamente realizar ese mismo diseño, así que me puse a ello.

En su propia documentación, tenían un código para empezar, y no me pareció nada difícil de implementar, a parte de todo lo que necesitaría para poder implementar la gráfica.

```
const labels = Utils.months({count: 7});
const data = {
  labels: labels,
  datasets: [{
    label: 'My First Dataset',
    data: [65, 59, 80, 81, 56, 55, 40],
    backgroundColor: [
      'rgba(255, 99, 132, 0.2)',
      'rgba(255, 159, 64, 0.2)',
      'rgba(255, 205, 86, 0.2)',
      'rgba(75, 192, 192, 0.2)',
      'rgba(54, 162, 235, 0.2)',
      'rgba(153, 102, 255, 0.2)',
      'rgba(201, 203, 207, 0.2)'
    ],
  }],
}
```

Pero a la hora de implementarlo, surgieron muchos inconvenientes.

Inconvenientes con Chart.js

El primer inconveniente que surgió fue que tuve que crear mi propia API para realizar las consultas necesarias. Chart.js no puede acceder directamente a la base de datos, por lo que la API debía encargarse de obtener los datos y devolverlos a la gráfica.

Debido a mi curiosidad, tuve que intentar realizarlo; me dispuse a leer información sobre cómo generar una API para que Chart.js pueda funcionar, y encontré información muy valiosa en su propia documentación.

Datos necesarios para la creación de la API.

Pensé que, con esto sería suficiente para poder crear una gráfica pequeña e ir trabajando, *pero me equivoqué*. Aunque la documentación está muy bien explicada, llevarlo a cabo dentro del sistema con consultas internas, hizo que fuera toda una odisea. Así que, me ayudé con ChatGPT.

Tras muchos prompts y muchas correcciones, me creó una API interna para que Chart.js pudiera ver los datos; y este fue el resultado:



Había quedado fenomenal, pero había un problema: el código —no lo había escrito yo— aunque hice bastantes correcciones que ChatGPT no generaba correctamente.

Intenté aprender para generar yo mi propia API y poder hacer una propia, pero me pareció demasiado tedioso y complicado, prácticamente como cursar un módulo entero de DAW solo para esa parte. Así que me dispuse a realizarlo de una manera no tan visual, aunque me fastidie.

Generación de estadísticas

He generado una tabla filtrando por fecha (*\$GET*) y poder ver los precios y el total correspondiente.

```
<!-- filtro fecha -->
<form action="estadisticas.php" method="GET" class="row g-3 mb-4 align-items-end">
  <div class="col-md-5">
    <label for="fecha_inicio" class="form-label">Fecha Inicio</label>
    <input type="date" class="form-control bg-dark text-white" id="fecha_inicio" name="fecha_inicio" value="<?php echo date('Y-m-01');>">
  </div>
  <div class="col-md-5">
    <label for="fecha_fin" class="form-label">Fecha Fin</label>
    <input type="date" class="form-control bg-dark text-white" id="fecha_fin" name="fecha_fin" value="<?php echo date('Y-m-d'); ?>">
  </div>
  <div class="col-md-2">
    <button type="submit" class="btn btn-primary w-100">
      <i class="fas fa-filter me-2"></i> Filtrar
    </button>
  </div>
</form>
```

Donde automáticamente se pone en el día 01 del mes actual en **Fecha Inicio** y en el día actual de **Fecha Fin**.

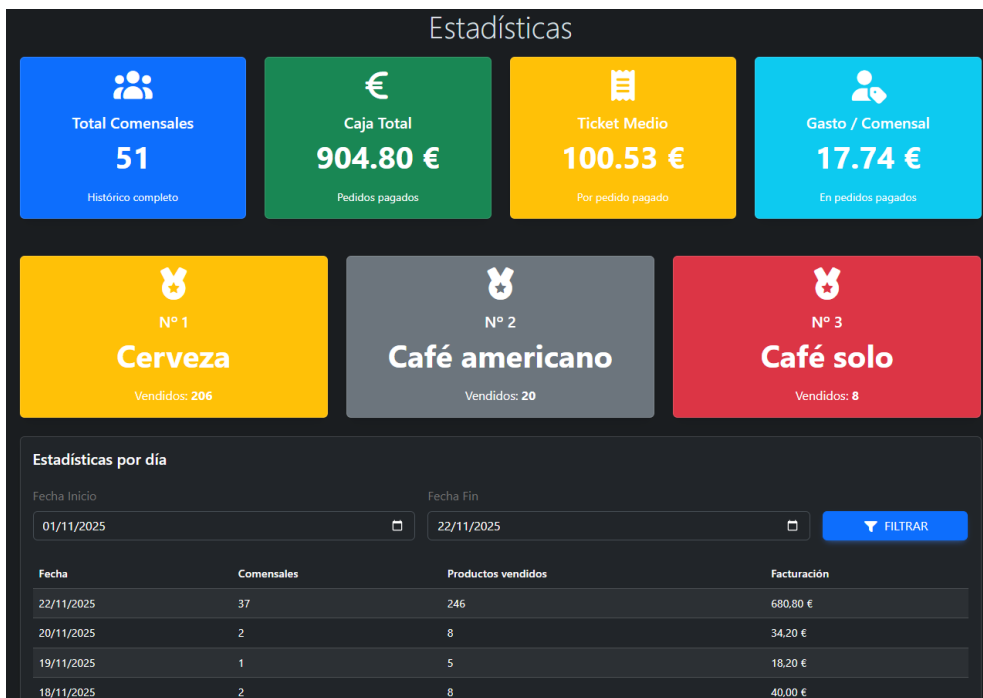
```
// todos los pedidos pagados y la fecha seleccionada por filtro
$consultaPedidos = "SELECT * FROM pedido WHERE estado = 2 AND DATE(fecha) >= '$fechaInicio' AND DATE(fecha) <= '$fechaFin'";
$resultadoPedidos = mysqli_query($conn, $consultaPedidos);

while ($pedido = mysqli_fetch_array($resultadoPedidos)) {
  $idPedido = $pedido['id'];

  // buscamos si este producto esta en este pedido
  $consultaProductoPedido = "SELECT cant FROM producto_pedido WHERE idPedido = '$idPedido' AND idProducto = '$idProducto'";
  $resultadoProductoPedido = mysqli_query($conn, $consultaProductoPedido);

  if ($rowProductoPedido = mysqli_fetch_array($resultadoProductoPedido)) {
    $cantidadTotal += $rowProductoPedido['cant'];
  }
}

$totalFacturado = $cantidadTotal * $precioProducto;
if ($cantidadTotal > 0) {
  echo "<tr>";
  echo "<td> $nombreProducto </td>";
  echo "<td> $nombreCategoria </td>";
  echo "<td> $cantidadTotal </td>";
  echo "<td> . number_format($precioProducto, 2) . " €</td>";
  echo "<td> . number_format($totalFacturado, 2) . " €</td>";
  echo "</tr>";
}
```



Top 3 productos más vendidos

Para el **top 3 productos**, he realizado **un bucle** por número de ventas, generando tres **cards** con tres colores.

Al meter todos las cards en el propio bucle, si no existen datos, simplemente no se generarán los cards.

```
<!-- top 3 productos más vendidos -->
<div class="row g-4 mb-4 mt-4">
  <?php
  // obtener productos más vendidos
  $consulta = "SELECT idProducto, SUM(cant) as total FROM producto_pedido WHERE idProducto > 0 GROUP BY
  idProducto ORDER BY total DESC";
  $resultado = mysqli_query($conn, $consulta);

  $posicion = 0;
  while ($row = mysqli_fetch_array($resultado)) {
    if ($posicion >= 3) {
      break;
    }

    $idProducto = $row['idProducto'];
    $cantidad = $row['total'];

    // obtener nombre del producto
    $consultaNombre = "SELECT nombre FROM producto WHERE id = '$idProducto'";
    $resultNombre = mysqli_query($conn, $consultaNombre);
    $rowNombre = mysqli_fetch_array($resultNombre);

    $nombre = $rowNombre['nombre'];
    $numero = $posicion + 1;
    if ($posicion == 0) {
      $color = "bg-primary";
    } elseif ($posicion == 1) {
      $color = "bg-warning";
    } else {
      $color = "bg-danger";
    }
  }
  ?>
  <div class="col-12 col-md-6 col-xl-4">
    <div class="card <?php echo $color; ?> text-white h-100">
      <div class="card-body text-center">
        <h5 class="card-title">
          <i class="fas fa-medal fa-2x mb-3"></i><br>
          N° <?php echo $numero; ?>
        </h5>
        <p class="card-text display-6 fw-bold">
          <?php echo $nombre; ?>
        </p>
        <p class="card-text">
          Vendidos: <strong><?php echo number_format($cantidad, 0); ?></strong>
        </p>
      </div>
    </div>
  </div>
  <?php
  $posicion++;
}
```



Nº 1

Cerveza

Vendidos: 34



Nº 2

Coca-Cola

Vendidos: 27



Nº 3

Agua mineral

Vendidos: 25

Tickets de cocina

Cuando el cliente realiza su pedido y le da a 'Pedir', le lleva a **ticketCocina.php**, donde generará un ticket con la información necesaria:

- Número de mesa
- Número del pedido
- Fecha del pedido
- Cantidades
- Nombre del producto
- Comentarios



```
$consultaLineas = mysqli_query(
    $conn,
    "SELECT *
    FROM producto_pedido
    WHERE idPedido = '$idPedido' AND servido = 0"
);

while ($linea = mysqli_fetch_assoc($consultaLineas)) {
    $idProducto = $linea["idProducto"];
    $cantidad = $linea["cant"];
    $comentario = trim($linea["comentario"] ?? "");

    $consultaProd = mysqli_query(
        $conn,
        "SELECT nombre FROM producto WHERE id = '$idProducto'"
    );

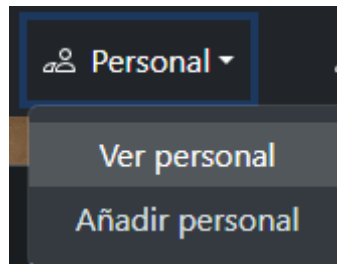
    $rowProducto = mysqli_fetch_array($consultaProd);

    // línea principal: " 2x  Croquetas jamon"
    $nombre = quitaTildes($rowProducto["nombre"]);
    $lineaProd = sprintf("%2dx  %s\n", $cantidad, $nombre);
    $printer->text($lineaProd);

    // si hay comentario, lo imprimimos en la línea de abajo, con tabs
    if ($comentario !== "") {
        $comentario = quitaTildes($comentario);
        $printer->setEmphasis(true);
        $printer->text("    -> " . $comentario . "\n");
        $printer->setEmphasis(false);
    }
}
```

Capacidades del rol Encargado

He implementado una sección del personal que es la siguiente:



En **personal.php** el encargado puede ver al personal registrado, bloqueándolo o editando sus datos.

Listado del personal						
DNI	Nombre	Apellido	Rol	Email	Editar	
2	Pedro	El camarero	Camarero	2@gmail.com	Editar	Bloquear

```
<?php
// realizo una tabla con todos los usuarios de la base de datos.
$consulta = "SELECT * FROM usuario WHERE rol = 2 OR rol = 1";

$result = mysqli_query($conn, $consulta);

while ($row = mysqli_fetch_array($result)) {
    $id = $row['dni'];
    $estado = $row["estado"];

    // listamos todos menos el que está consultando actualmente
    if ($usuario != $row["nombre"]) {
        if ($estado == 0) {
            print("<tr>");
        } else {
            print("<tr class='table-danger'>");
        }
        print("<td class='d-none d-lg-table-cell'>");
        print($row['dni']);
        print("</td>");

        print("<td>");
        print($row['nombre']);
        print("</td>");

        print("<td>");
        print($row['apellido']);
        print("</td>");
```

Si se bloquea, esto llevará por **\$GET** a un archivo llamado **bloquearPersonal.php** que simplemente cambia el estado a 1 para bloquear o 0 para desbloquear.

```
// funcionalidad para bloquear el producto
include("../components/conexion.php");
include("seguridad.php");

$id = $_GET["id"];

$consulta = "UPDATE usuario
SET estado = 1
WHERE dni = '$id'";

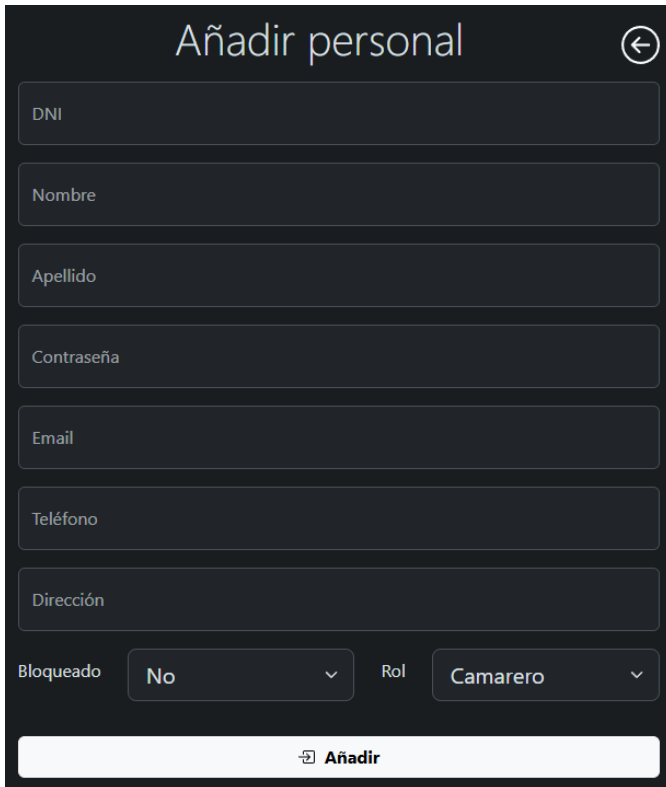
$result = mysqli_query($conn, $consulta);

mysqli_close($conn);

header("LOCATION: /restaurante/encargado/personal.php");
exit;
```

Añadir personal

Para añadir personal, he realizado un formulario básico en **addPersonal.php**, donde se necesita poner los datos del personal asociado y el rol pertinente que este tenga, con la opción de bloquearlo o no bloquearlo en caso de que se incorpore más adelante, tenerlo bloqueado.



```
if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $dni = $_POST["dni"];
    $nombre = $_POST["nombre"];
    $apellido = $_POST["apellido"];
    $email = $_POST["email"];
    $telefono = $_POST["telefono"];
    $direccion = $_POST["direccion"];
    $estado = $_POST["bloqueado"];
    $rol = $_POST["rol"];
    $pw = $_POST["pw"];

    if ($estado == "si") {
        $bloqueado = 0;
    } else {
        $bloqueado = 1;
    }

    if ($rol == "camarero") {
        $rol = 1;
    } else if ($rol == "encargado") {
        $rol = 2;
    }

    if ($direccion == "") {
        $direccion = NULL;
    }
}
```

```
$consulta = "INSERT INTO usuario (dni, password, nombre, apellido, rol, email, telefono, direccion, estado)
VALUE ('$dni', '$pw', '$nombre', '$apellido', '$rol', '$email', '$telefono', '$direccion', '$estado')";

mysqli_query($conn, $consulta);
```

Añadidos extra

Cada rol permite acceder a la página **perfil.php**, una página donde se ven sus credenciales y pueden editarlas, **excluyendo el DNI**, que está en **'readonly'**.

Editar perfil

Encargado

DNI
1

Nombre *
Álvaro

Apellido *
Martínez

Contraseña *
•

Email *
alvaro@gmail.com

Teléfono *
666666666

Dirección

 **Editar**

Editar perfil

Encargado

Has cambiado los datos con éxito

DNI
1

Nombre *
Álvaro

Apellido *
Martínez

Contraseña *
•

Email *
alvaro@elquintopino.com

Teléfono *
666666666

Dirección

 **Editar**

Conclusión

Esta práctica me ha servido mucho para reforzar la lógica de programación de PHP y bases de datos. Una vez entendida la estructura del proyecto, desarrollar el sistema del restaurante me ha ayudado a afianzar todo lo aprendido en la **PokeDAW**.

Me ha gustado poder tomar mis propias decisiones en cuanto al diseño y la organización interna, porque te da la sensación de que realmente puedes construir algo completo desde cero. Además, trabajar por Sprints ha hecho que el desarrollo sea mucho más llevadero y ordenado, y he podido comprobar cómo se trabaja en realidad en este sector.

En general, esta práctica me ha permitido aplicar conocimientos reales de servidor, roles, control de sesiones y gestión de datos, y siento que he ganado confianza para enfrentar proyectos más grandes.

Bibliografía

[Iconos de Bootstrap](#)

[Iconos de Fontawesome](#)

[Remover fondo de las fotos de los productos](#)

[Quitar marca de agua de las fotos de los productos](#)

[Chart.js](#)

[MDBootstrap](#)